

CPT202 Assignment 3

Final Report for Software Engineering Group Project

Project B: Music Resource Management

Melodify - A secure, personalized and user-friendly system

2025/05/21

<http://101.37.37.8:8080>

Student Name : Xu Chen

Student ID : 2257453

Group 7 Members:

1. Chenrui Ding, 2252734
2. Jin Zhou, 2251739
3. Xinyi Yang, 2251657
4. Xu Chen, 2257453
5. Xuexin Zhan, 2251792
6. Yiyi Zhu, 2252441
7. Youlong Xu, 2252978
8. Zichen Qiu, 2252705

Contents

1	Introduction	1
2	Software Development Process	2
2.1	Individual Sprint Backlog	2
2.2	The Ways that Scrum facilitates Software Development and Effective Communication	2
2.2.1	Iterative Development for Rapid Value Validation	2
2.2.2	Transparent Communication through Daily Scrum	3
2.2.3	Adaptability to Change and Flexible Plan Adjustment	3
3	Software Design	3
3.1	Overview of Individually Developed PBIs	3
3.2	Detailed Design and Implementation	4
3.2.1	Sprint 1 : Admin content review	4
3.2.2	Sprint 2 : Music Resource/Account Lock/Unlock Functionality	5
3.2.3	Sprint 3 : Usability Optimization and Feedback System	6
4	Change management	7
5	Legal, social, ethical, and professional issues	8
6	Conclusion and Future Work	8
	Appendix A: Evidences on Software Development Process	10
	Appendix B: Sequence Diagram UML about Sprint 1	10
	Appendix C: ER Diagram	12
	Appendix D: Main Code about Admin	13
	Appendix E: Main Code about Feedback	21
	Appendix F : User Admin Sequence Diagram	24
	Appendix G: Secure encryption	25
	Appendix H: PBIs and Artifacts	26
A.1	PBI 7.1: Admin Review of User-Uploaded Resources	26
A.2	PBI 7.2: Block Unauthorized/Prohibited Content	26
A.3	PBI 7.3: Lock and Unlock User Accounts	26
A.4	PBI 7.4: View All Music Resources	26
A.5	PBI 7.5: Manage User Accounts	26

1 Introduction

Driven by the wave of digitalization, the field of digital music has developed rapidly, and the way of acquiring, storing and sharing music resources has undergone tremendous changes [1], the development of music resource management systems has become increasingly important. However, the user authentication process of some platforms is cumbersome, which affects the efficiency of use; the music upload and organization functions are not smart enough, which makes it difficult for users to quickly organize the huge music library [2]. At the system management level, administrators lack efficient tools for content review and management, and user privacy and data security also face potential risks, such as data leakage and illegal access [2]. The existing solutions mainly include optimizing the user interface design, simplifying the authentication process, improving the accuracy of music classification and search; strengthening data encryption technology to ensure user information security [3]. Nevertheless, these methods lack systematic integration and are difficult to fundamentally meet the diverse needs of users and administrators.

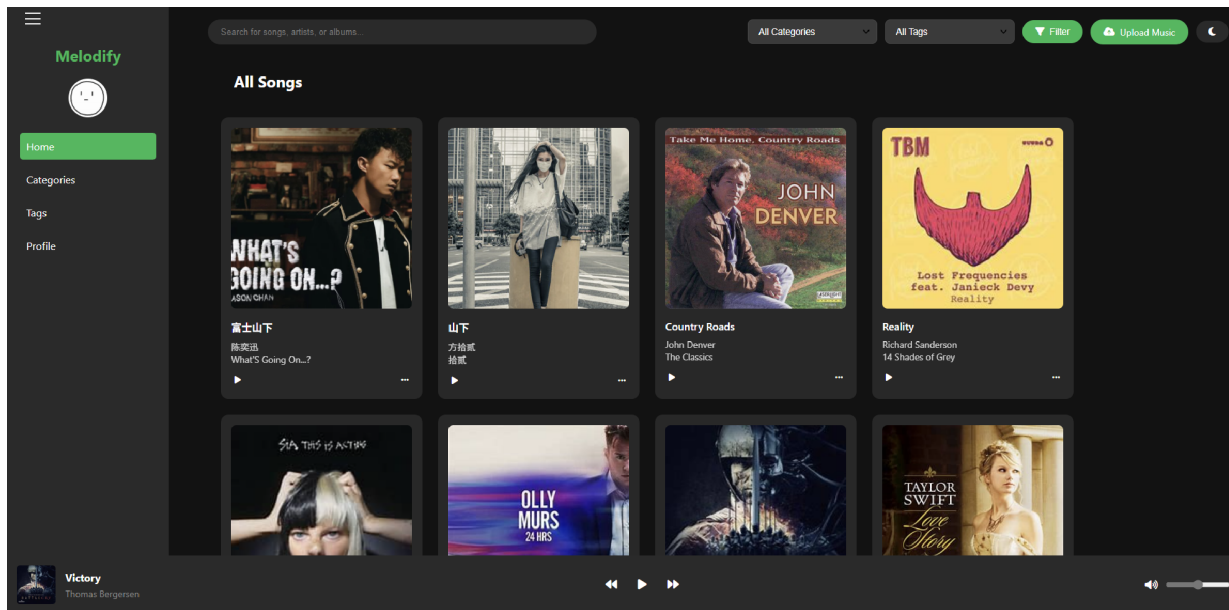


Figure 1.1: The User Interaction Interface of the Music Management System (Melodify)

This project aims to develop Melodify, a comprehensive and user-friendly music resource management system. The system prioritizes secure and convenient user authentication, enabling seamless access for users. It offers robust music upload and organization capabilities, supporting automatic tag and category generation to facilitate efficient content classification and quick retrieval. For system management, administrators are provided with visual interfaces to review music uploads and playback metrics, as well as tools to block or unblock users. On the playback front, Melodify supports essential functionalities such as music streaming, volume control, and song navigation (forward/backward), ensuring a smooth listening experience for users.

In this project, my own work includes :

- I implemented a fully functional backend administrator control interface, which provides comprehensive backend management functions for the music resource management platform. This controller is implemented through the Spring MVC framework and adopts a role-based access control (RBAC) design to ensure that only administrators can manage users, audit resources and manage the administrator's personal center.
- On the front end, I implement user information status filtering, batch operations, playback preview and theme switching for administrators.
- Add additional user feedback submission, write relevant test code, help other members modify error code, integrate related code, and write group report.

This report will begin by discussing the Scrum framework role in enabling iterative development and cross-functional collaboration, with a focus on the author contributions to user management and audit modules. The Software Design section details the architecture and implementation of individual PBIs, using UML diagrams and code examples to explain role-based access control, music auditing workflows, and user profile management, while justifying technical choices and referencing appendices for test cases and screenshots. A Change Management section illustrates adaptability through feature update example, demonstrating Scrum flexibility. The report also addresses legal, social, ethical, and professional considerations, emphasizing data privacy, content compliance, and code quality. Finally, the conclusion summarizes lessons learned and proposes future improvements.

2 Software Development Process

During the software development phase, our team follows the Scrum framework. Scrum is an Agile framework designed to iteratively deliver value through time-boxed cycles called Sprints. The team prioritizes product backlog items, which break down user-centric requirements into actionable tasks and selects high-priority PBIs for each Sprint. For more evidences, see Appendix A.

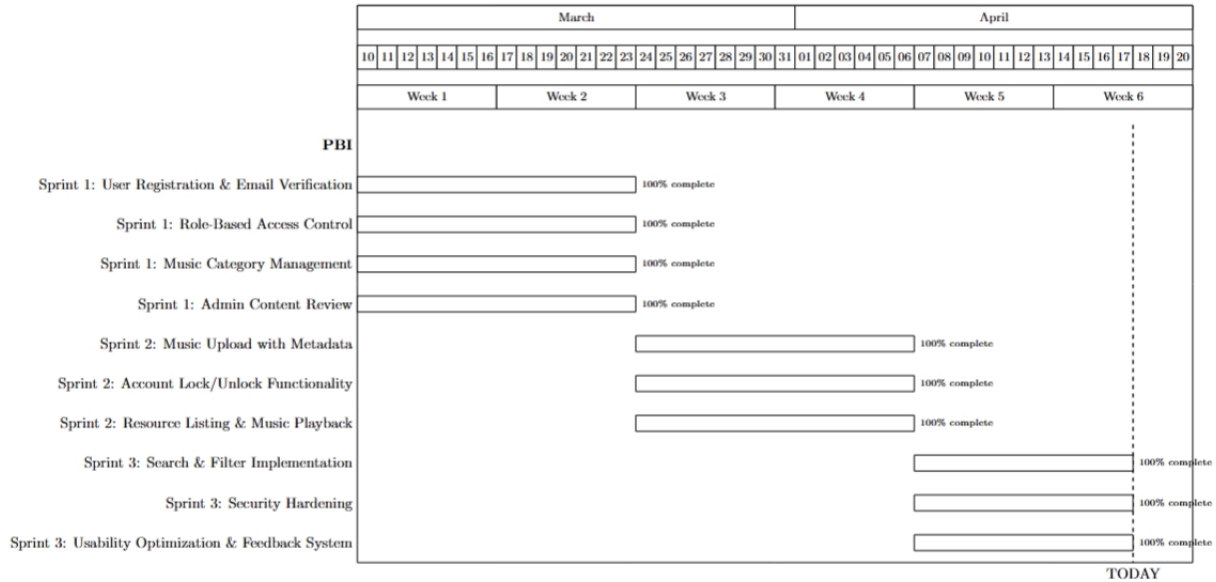


Figure 2.1: A Gantt chart of the product backlog at different sprint stages from CW2

2.1 Individual Sprint Backlog

At the beginning of the project, under the Scrum framework, I was assigned tasks to implement administrator related functions. I formulated the following Product Backlog Items, which comes from CW1 :

- PBI 7.1: Admin Review of User - Uploaded Resources
- PBI 7.2: Block Unauthorized/Prohibited Content
- PBI 7.3: Lock and Unlock User Accounts
- PBI 7.4: View All Music Resources
- PBI 7.5: Manage User Accounts

As can be seen from the Gantt chart, during the three iterations: in the first iteration, I was responsible for presenting the administrator interface, which involved PBIs 7.1, 7.4, and 7.5. In the second iteration, my tasks centered around user account locking and unlocking as well as approving music uploads, which involved PBIs 7.2 and 7.3. In the third iteration, I focused on user - feasibility optimization and the user feedback system.

2.2 The Ways that Scrum facilitates Software Development and Effective Communication

From my perspective, Scrum has facilitated the software development process in the following three ways.

2.2.1 Iterative Development for Rapid Value Validation

Scrum places a strong emphasis on iterative development, advocating for the decomposition of large-scale projects into smaller, more manageable iterations rather than attempting to develop them in a single monolithic phase. Take the development of administrator-related functions as an instance: during the first iteration, our focus was on implementing the basic display of the administrator interface (PBI 7.1, 7.4, 7.5). This approach enabled us to collaborate with other team members to conduct the first round of integration testing for functionalities such as login, playback, and the administrator interface. Through this process, we collectively defined the information structure within each class and

clarified the design of database tables, ultimately showcasing a preliminary prototype. This iterative method not only accelerated the validation of core functionalities but also established a solid foundation for subsequent development by resolving structural inconsistencies at an early stage.

2.2.2 Transparent Communication through Daily Scrum

We follow the Scrum framework and hold daily stand-up meetings. This open forum style of communication means that any issues can be resolved quickly. For example, I am developing features related to user account locking and unlocking in PBI 7.3, but my other team members have not yet written the structure of the user class, which directly affects my back-end user database. We can communicate to determine the information contained in the class, and other team members can immediately provide suggestions or help. In this way, information does not exist in isolation among team members or sub-teams. It ensures that everyone understands the progress of the project and any potential bottlenecks are resolved in a timely manner, thereby reducing information gaps and improving the overall efficiency of the team.

2.2.3 Adaptability to Change and Flexible Plan Adjustment

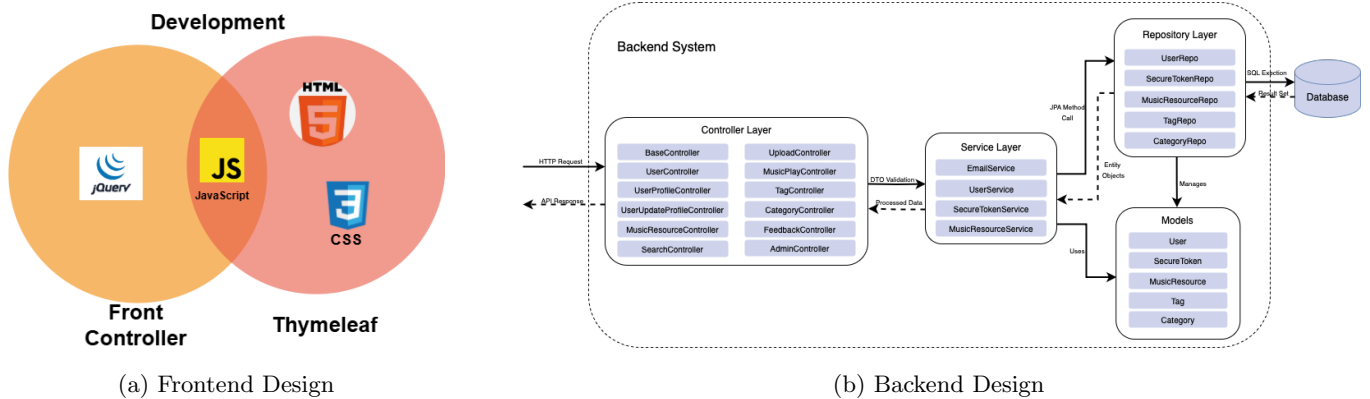
One of Scrum's most significant strengths lies in its inherent adaptability to change. As evidenced by the initial five PBIs, the user feedback feature was not initially included in the project scope, yet it was successfully implemented in the final Gantt chart. This outcome arose from a scenario in which, during project execution, I identified that team members were unable to complete tasks related to the user feedback system. Rather than allowing project delays or feature omissions, Scrum empowered me to take proactive measures. Specifically, I introduced

- PBI 7.6 User feedback capabilities

The flexibility of the framework stems from the dynamic nature of the Product Backlog, which makes such adjustments possible, allowing for adjustments in priorities and integration of new requirements without affecting the project schedule.

3 Software Design

3.1 Overview of Individually Developed PBIs



The following is the PBI I completed:

- PBI 7.1: Admin Review of User - Uploaded Resources
- PBI 7.2: Block Unauthorized/Prohibited Content
- PBI 7.3: Lock and Unlock User Accounts
- PBI 7.4: View All Music Resources
- PBI 7.5: Manage User Accounts
- PBI 7.6 User feedback capabilities

In the frontend system, I use the Thymeleaf template engine and native JavaScript to implement dynamic rendering and interaction logic of the administrator interface, focusing on the following core functions:

- I use Thymeleaf to traverse the MusicResource list returned by the backend and dynamically generate review cards. For example, the conditional expression `th:if="music.status==1"` controls the "Approve" button to be displayed only when the resource is in the "Awaiting Review" status in PBI 7.1 and PBI 7.4.
- I use JavaScript to listen for button click events, trigger form submission (such as `/admin/users/lock`), and use Thymeleaf to render user status labels (such as the red "Locked" logo) in PBI 7.3 and PBI 7.2.
- The feedback content is collected and submitted to the back-end interface `/feedback` through a POST request, using native JavaScript to implement simple input verification in PBI 7.6.

The backend system employs a layered architecture to implement core functionalities. In the controller layer, AdminController postAuditPage handles audit requests by verifying administrator permissions via HttpSession and invoking MusicResourceService to update resource statuses. The lockUser method checks user roles (preventing admin locks) and uses UserService to toggle user status to "locked". The service layer MusicResourceService manages business logic: when a resource is marked "banned", it reduces the uploader banCount, dissociates the resource from categories/tags, and updates category/tag heat values, all within a transaction (@Transactional) for data consistency. FeedbackService validates and stores feedback, linking it to user IDs for tracking. In the repository layer, MusicResourceRepo uses findByStatus with pagination for efficient admin resource filtering, while UserRepo leverages findByRole to retrieve regular users, supporting status-based sorting. The model layer defines MusicResource and User entities, with fields like status and banCount driving lifecycle and access control logic.

3.2 Detailed Design and Implementation

3.2.1 Sprint 1 : Admin content review

In terms of **Logic Design and Implementation**, the core goal of Sprint 1 is to achieve comprehensive control of system content by administrators, covering three major functions: resource review (PBI 7.1), resource viewing (PBI 7.4), and user account management (PBI 7.5). These three PBIs receive front-end requests through the controller layer, the service layer processes the core business logic, the data access layer completes database operations, and finally presents the results through the view layer. In terms of logical flow, the review process of PBI 7.1 (Admin Review of User - Uploaded Resources) begins with the administrator selecting the resource to be reviewed through the UI. After the controller verifies the permissions, it calls the service layer to update the resource status (such as from "pending review" to "passed"), and triggers related operations (such as updating user upload statistics and sending notifications). PBI 7.4 (View All Music Resources) receives query parameters (such as category, status) through the controller, and the service layer calls the data access layer to obtain the resource list, which is finally displayed in pages by the view layer. The process of PBI 7.5 (Manage User Accounts) is similar. The administrator initiates a user query or operation request, and the system verifies the permissions and executes the corresponding user information modification or status change.

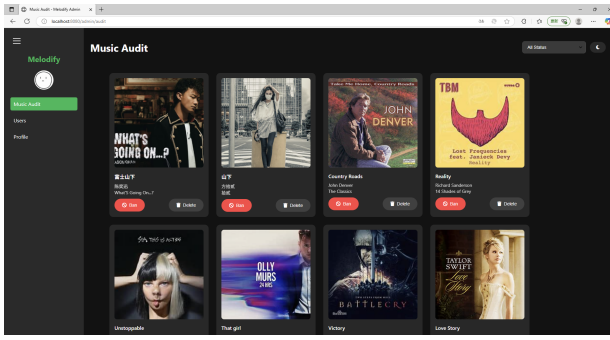
Please go to the Appendix B to see the Sequence Diagram UML.

In terms of **Database Design** for the music resource management system centers on two core entities: MusicResource and User, structured to support key PBIs efficiently. The MusicResource entity uses a status field (0-approved, 1-pending, 2-banned) to manage resource lifecycles, with ManyToOne relationships to User, Category, and Tag for tracking uploaders and categorization. This supports PBI 7.1 review process by enabling status updates and associating resources with uploaders to update user statistics (e.g., uploadCount, approvedCount) in atomic transactions. For PBI 7.4 (viewing resources), composite indexing on category_id and status, along with database views for pre-computing popular resource metrics, optimizes query performance for filtered and sorted results. The User entity role (0-admin, 1-user) and status (0-verified, 1-unverified, 2-banned) fields enforce access control and account management. In PBI 7.5, locking a user (status=2) triggers a query via the user relationship in MusicResource to hide all resources by the locked user, ensuring consistent access restrictions across entities. This design balances data integrity, query efficiency, and lifecycle management for system scalability and compliance with PBIs.

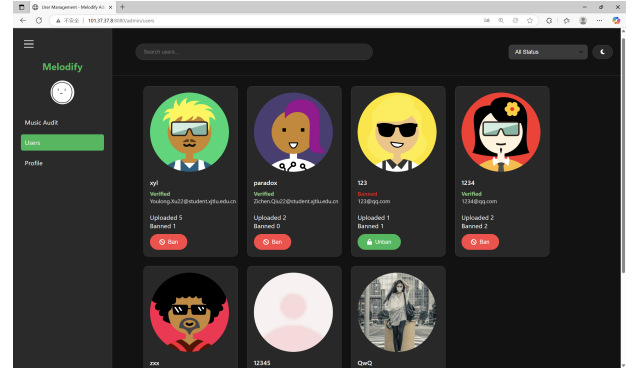
Please go to the Appendix C to see the ER Diagram.

In terms of **MusicResource Object States**, When a resource is initially uploaded, its status is set to 1 (pending review). In this state, the resource is only visible to the uploader, and it is added to the administrator's review queue. Upon review, if the resource is approved, its status is changed to 0 (approved). This triggers several actions: the resource becomes publicly visible, relevant category and tag statistics are updated and the uploader's approvedCount (in the User entity) is incremented. On the other hand, if the resource is rejected, the status is set to 2 (banned). The resource is then marked as invisible, and the uploader's rejectedCount in the User entity increases. Repeated rejections may lead to the user's account being flagged or even locked, as per the system's rules.

In terms of **User Object State**, it is closely intertwined with the review outcomes of their uploaded resources. When a user's rejectedCount reaches a threshold (e.g., rejectedCount $\geq$ 3), the system automatically changes the user's status to 2 (banned). The user is prohibited from uploading new resources, and all their existing resources are temporarily hidden by querying and updating the related MusicResource entities. Administrators can use the functionality in PBI 7.5 to manually unlock a user account by changing the status back to 0 (verified). This restores the user's normal permissions, allowing them to resume uploading and accessing their resources.



(a) Admin View All Music Resources



(b) Admin View All Users

In terms of **UI design**, the user interface serves three PBIs at the same time through modular design. The top navigation bar provides three entrances: "Music Review", "User Management" and "Administrator Information", and each entrance corresponds to an independent functional module. The music review page adopts a card-style layout. Each card displays a resource thumbnail, basic information and status label (red - pending review, green - passed). The bottom of the card integrates "Pass" and "Reject" shortcut buttons to support batch operations. The user management interface also adopts a table layout, focusing on displaying user status (normal/locked), upload statistics and operation buttons. Click the user name to view detailed information and adjust permissions. Specifically

- For PBI 7.1, the card-style layout of the music review page significantly simplifies the review process. Color-coded status labels provide immediate visual feedback, allowing administrators to quickly identify the status of each asset without reading detailed information. This reduces cognitive load and speeds up the review decision process. Batch operations are supported, allowing administrators to process multiple assets in a single workflow without navigating between individual asset pages. This not only saves time, but also minimizes the possibility of errors.
- For PBI 7.5, the tabular layout of the user management interface presents key information in a clear and organized manner. By prominently displaying user status (active/locked), upload statistics, and action buttons, administrators can easily understand the overall status of user accounts at a glance. Clicking a username to access detailed information and adjust permissions directly from the interface simplifies the account management process. This design reduces the steps required to perform tasks and modifying user roles.

3.2.2 Sprint 2 : Music Resource/Account Lock/Unlock Functionality

In terms of **Logic Design and Implementation**, the implementation of PBI 7.2 and PBI 7.3 was centered around enhancing security and user management capabilities. The design adopted a layered architecture approach, with the controller layer handling incoming requests, the service layer implementing business logic, and the repository layer managing database interactions.

- For PBI 7.2, when the system detects unauthorized or prohibited content, the controller receives the alert signal. The service layer then validates the nature of the content by cross-referencing with a predefined list of prohibited keywords, file types, or patterns. If the content is deemed prohibited, the service layer updates the relevant MusicResource entity's status to "blocked" (represented by a specific integer value, e.g., 2). It also notifies the user associated with the content upload, records the incident in an audit log table, and may trigger additional actions such as reducing the user's credibility score.
- For PBI 7.3, the account lock/unlock functionality was implemented with a focus on security and flexibility. When an administrator initiates a lock or unlock action from the user management interface, the request is sent to the controller. The controller first verifies the administrator's Permissions through Spring Security's role-based access control mechanisms. The service layer then retrieves the target User entity from the database using the UserRepository. Based on the action (lock or unlock), it updates the User entity's status field (0 for active, 2 for locked). If a user is locked, the service layer also ensures that any ongoing sessions of that user are invalidated, and access to all resources uploaded by the user is restricted.

In terms of **Database Design**, by changing the status to indicate that content is blocked, the system can easily filter and manage prohibited resources. Additionally, the relationship between the MusicResource and User entities allows the system to associate prohibited content with its uploader. For PBI 7.3, the status field directly controls the user's access rights. When a user is locked (status = 2), the application can query the User table to identify locked users and enforce access restrictions. The role field is used to ensure that only administrators (with role = 0) have the permission to perform lock and unlock operations.

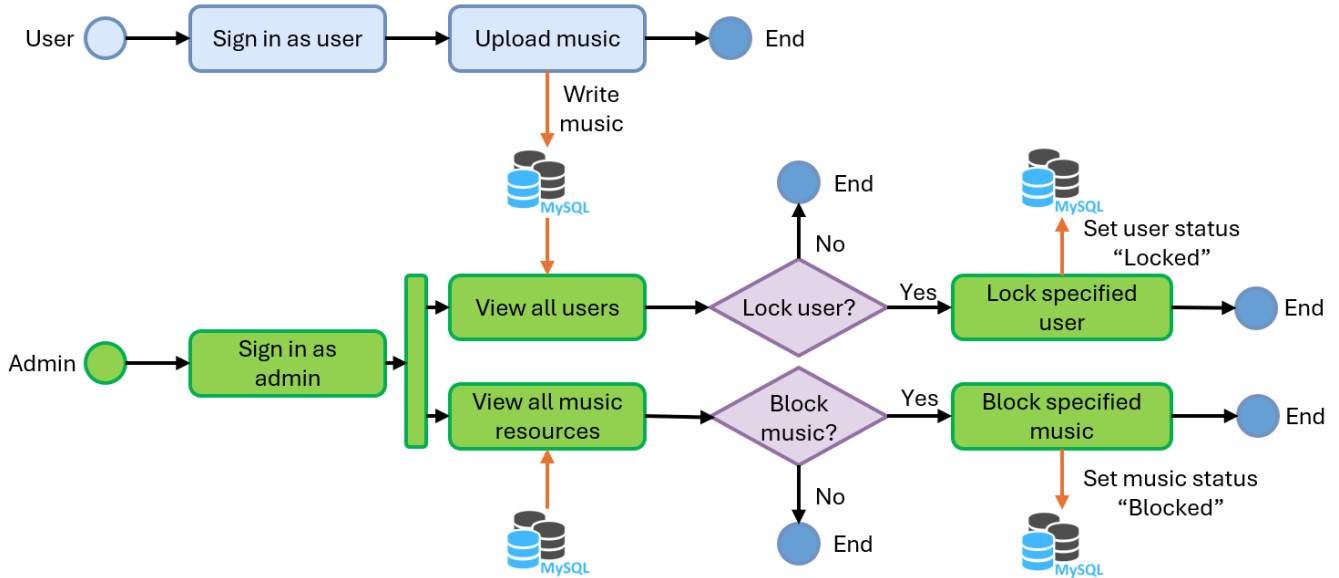


Figure 3.3: Admin Lock user/Block music process

In terms of **MusicResource Object States**, it can transition from a normal state (e.g., approved with status = 0) to a blocked state (status = 2). When the content is blocked, its visibility is restricted. It is no longer accessible to regular users, and search results will exclude blocked resources. This change also has a cascading effect on the associated User object. The user's reputation may be affected, and repeated uploads of prohibited content could lead to their account being locked in PBI 7.3.

In terms of **User Object State**, it has two main states relevant to the functionality: active (status = 0) and locked (status = 2). In the active state, users can access all permitted features, upload resources, and interact with the system. When the user is locked, their authentication tokens are invalidated, preventing them from logging in. All their existing sessions are terminated, and any attempts to access resources or perform actions will result in access - denied errors. This state change effectively restricts the user's activities and serves as a disciplinary or security measure.

In terms of **UI design**

- For PBI 7.2, the content management interface includes a dedicated section for reviewing potentially prohibited content. Suspicious resources are highlighted with a distinct warning icon, and a detailed panel shows the reasons for the suspicion. Administrators can then easily review the content and use intuitive toggle buttons to block or unblock it. The interface also provides a history log section, which enable administrators to track and audit content management activities.
- For PBI 7.3, the user management UI prominently displays a list of users with their current status clearly indicated using color coding. When an administrator hovers over these buttons, a tooltip appears, providing additional information about the implications of the action. After a lock or unlock action is performed, the UI immediately refreshes to reflect the updated status, giving real - time feedback. The design also includes a search and filtering feature, allowing administrators to quickly locate specific users and perform batch operations, significantly improving the efficiency of account management tasks.

3.2.3 Sprint 3 : Usability Optimization and Feedback System

In terms of **Logic Design and Implementation**, the implementation of PBI 7.6, "User feedback capabilities," integrates frontend and backend components to enable seamless feedback submission and handling. On the backend, the FeedbackController serves as the central hub for processing requests. When a user accesses the /feedback endpoint via a GET request, the controller checks the user's role from the HttpSession. On the frontend, the feedback form is embedded within the user profile page.

In terms of **Database Design**, the Feedback entity contains the following fields: id (primary key), message (a text field to store the feedback content), type (an enumeration of categories such as "bug", "feature", "other", etc.), userId (a foreign key referencing the User entity to associate the feedback with the submitter), submissionTime (a timestamp to record when the feedback was sent), and status (to track whether the feedback has been reviewed, such as "pending", "in progress", "completed"). This design allows for efficient storage and retrieval of feedback data. For example, an administrator can query the Feedback table to retrieve all pending feedback. The userId foreign key can

be used to analyze feedback trends for each user, such as identifying users who frequently submit valuable suggestions or report bugs.

In terms of **States**, when the user successfully submits feedback, the user can see a prompt in the user interface that the feedback has been successfully submitted, and the feedback record will be displayed as "Submitted"

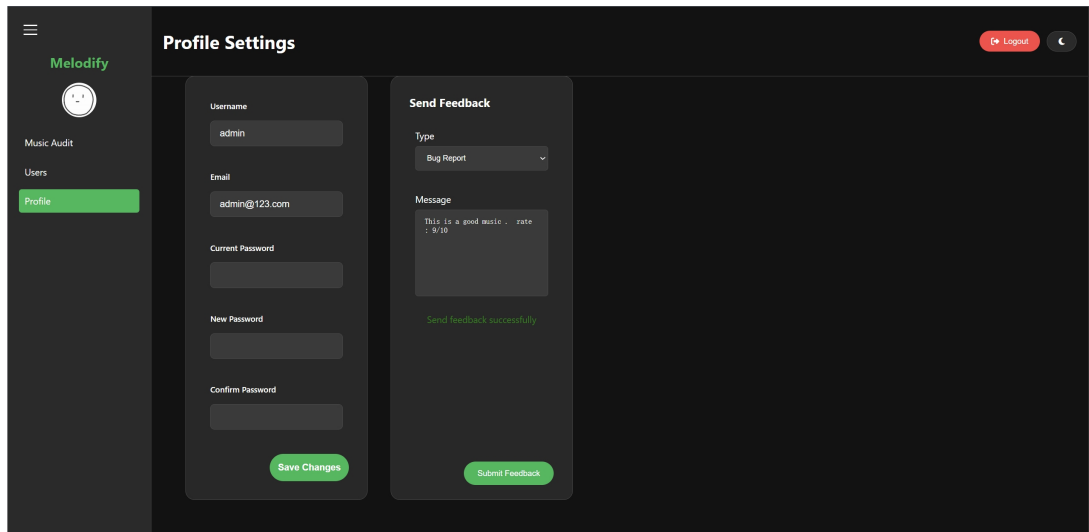


Figure 3.4: Feedback sent successfully

In terms of **UI design** for the feedback functionality is integrated into the user profile page, making it easily accessible and reducing the learning curve for users. The form layout is intuitive, with clearly labeled fields for "Type" and "Message". The use of a dropdown for the type field restricts input to predefined categories ("bug", "feature", "other"), ensuring data consistency and making it easier for administrators to categorize and prioritize feedback. Visual cues, such as the success and error messages displayed conditionally using Thymeleaf (`th:if="feedbackMessage"` and `th:if="feedbackError"`), provide immediate feedback to users. These messages are prominently placed on the page, ensuring that users are aware of the submission status without having to search for confirmation. The "Submit Feedback" button is clearly visible and styled distinctively, guiding users towards completing the action.

4 Change management

Change management is a critical part of any Scrum project, as it ensures that the project can adapt to changing requirements while maintaining the integrity of the project and creating value for stakeholders. In our project, the requirements for the Admin Information interface changed significantly, and it was initially limited in terms of navigation and functionality. The original design of the Admin Information interface allowed administrators to click on their avatars to access separate pages for modifying their avatars and emails. However, this design had some flaws. There was no clear way to return to the previous page except to manually modify the URL, which was tedious and error-prone.

To address these issues, we decided to introduce a navigation bar that would allow easy access to all the main parts of the application. The navigation bar would contain links to Music Management, User Management, and Admin Information, and administrators could switch between these parts with a single click. In addition, we improved the UI design of the Admin Information interface to make it more visually appealing and user-friendly. We added clear labels and instructions, as well as a more intuitive layout, to make it easier for administrators to find the information they need and perform necessary actions.

The process of implementing this change consisted of several steps. First, I identified the need for the change and documented the need in user stories. We then prioritized the user stories and added them to the product backlog. During the sprint planning meeting, I estimated the effort required to implement the change. The development team then began the implementation work. I first created a mockup of the new navigation bar and UI design to ensure that it met the requirements and was visually appealing. I then developed the necessary code to implement the navigation bar and integrate it with the existing application. They also made the necessary changes to the admin information interface to improve its functionality and usability. Once the development was complete, I also needed to test the new functionality to ensure that the navigation bar worked as expected and that the admin information interface was easy to use and navigate. I also tested the UI design to ensure that it was visually appealing and met accessibility standards. Once testing was complete, the changes were deployed to the production environment. We provided training to the administrators on how to use the new navigation bar and UI design to ensure that they could easily adapt to the change.

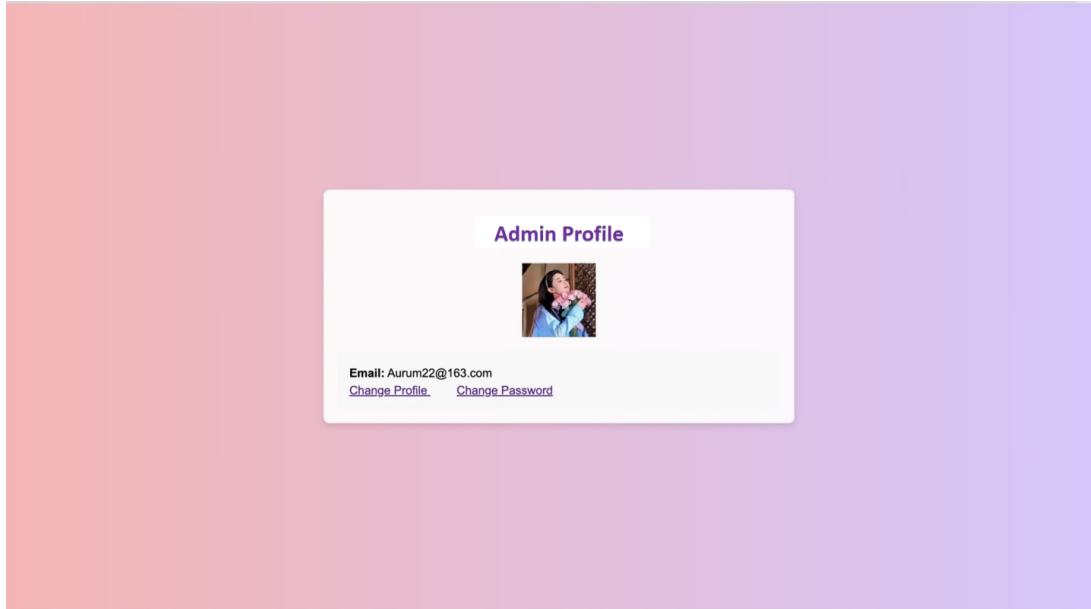


Figure 4.1: Previous Admin Profile

5 Legal, social, ethical, and professional issues

In terms of Legal Compliance Considerations, One issue is the potential violation of data privacy regulations (e.g., GDPR) in handling user data for feedback (PBI 7.6) and account management (PBI 7.3), such as storing emails and feedback content without explicit consent. One solution is to encrypt sensitive data fields (e.g., passwords in the User model) and embed a mandatory consent checkbox in the feedback form, stating that data is used solely for service improvement.

In terms of Social and Ethical Implications, One issue is the risk of bias in content moderation (PBI 7.1), where subjective admin decisions might disproportionately target certain content types or users. One solution is to implement automated content scanning to supplement manual reviews and establish clear, written guidelines for prohibited content, reducing human bias in decision-making. Another issue is that excessive blocking of legitimate content in PBI 7.2 may cause user dissatisfaction and limit content diversity. One solution is to adopt a "soft ban" policy that temporarily stores blocked content and opens appeals within 30 days, combined with a user feedback mechanism (PBI 7.6) to allow users to object to decisions and request reviews, thereby promoting a more inclusive governance process.

In terms of professional responsibilities and system governance, one issue is the lack of transparency in administrative actions, such as inexplicable account lockouts (PBI 7.3) or arbitrary content rejections (PBI 7.1), which can erode user trust. One solution is to maintain a detailed audit log of all review and account management actions, recording timestamps, operator IDs, and reasons, and enforce role-based access control (RBAC) through the role field of the user model to ensure that only authorized administrators (e.g., role=0) can perform sensitive actions.

6 Conclusion and Future Work

The application successfully implements robust security features to protect user data and enables personalized music uploads with tag-based categorization for efficient organization and discovery. Resource moderation tools and user feedback channels create a user-centric management environment, validated by stable real-world performance.

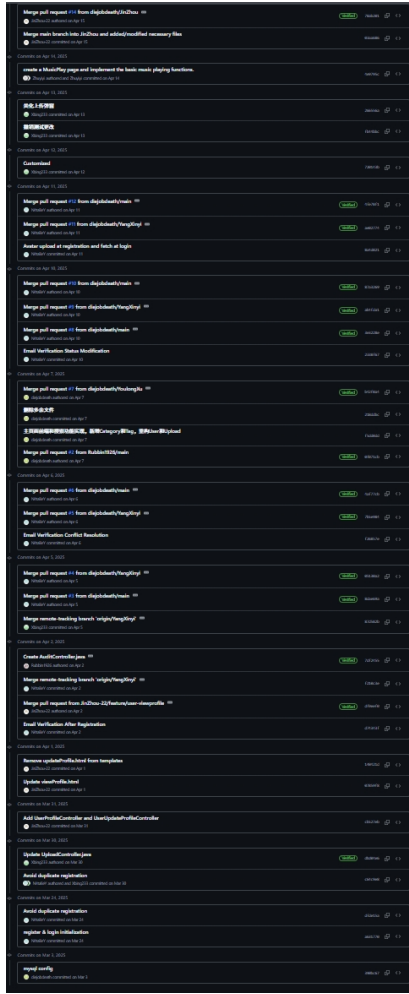
Through this project, I gained a deep practical understanding of agile development methods (Scrum), especially in managing iteration requirements and cross-functional collaboration. I also learned how to balance functionality with security and compliance, such as implementing email validation for user authentication and designing audit logs to meet legal requirements. The experience of addressing UI/UX limitations highlighted the value of user-centered design principles and iterative feedback loops. Technical lessons included optimizing database performance for PBI 7.4 through composite indexes and materialized views, and creating responsive server-rendered interfaces with Thymeleaf and Spring MVC.

However, the system faces two key limitations: basic digital integration strategies restricting commercial expansion and limited UI explainability, which hinders usability for diverse user groups, particularly in specialized contexts. To address these, future plans include adopting advanced digital integration with e-commerce and streaming platforms to enhance commercial viability, and redesigning the UI with intuitive visual cues, interactive tutorials, and guidance mechanisms to improve accessibility and user experience across all user segments. These initiatives aim to overcome current constraints and elevate the application overall functionality and market competitiveness.

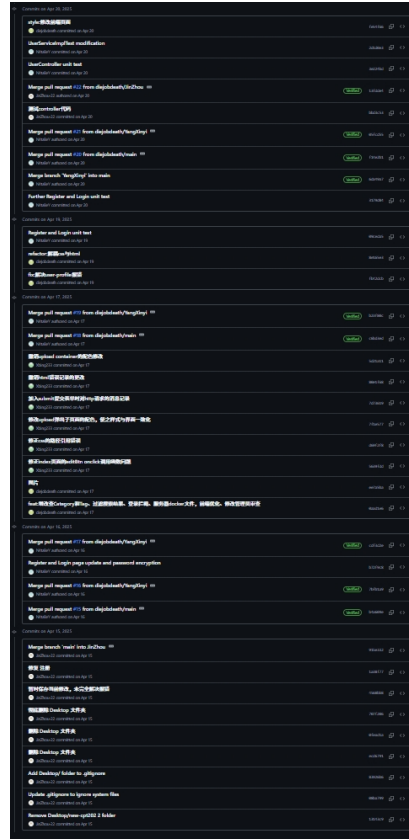
References

- [1] W. Liu, “Integrating data mining with internet of things to build the music project management system,” *Journal of Ambient Intelligence and Humanized Computing*, pp. 1–13, 2020.
- [2] E. Pizzolitto, “Music in business and management studies: a systematic literature review and research agenda,” *Management Review Quarterly*, vol. 74, no. 3, pp. 1439–1472, 2024.
- [3] D. Afchar, A. Melchiorre, M. Schedl, R. Hennequin, E. Epure, and M. Moussallam, “Explainability in music recommender systems,” *AI Magazine*, vol. 43, no. 2, pp. 190–208, 2022.

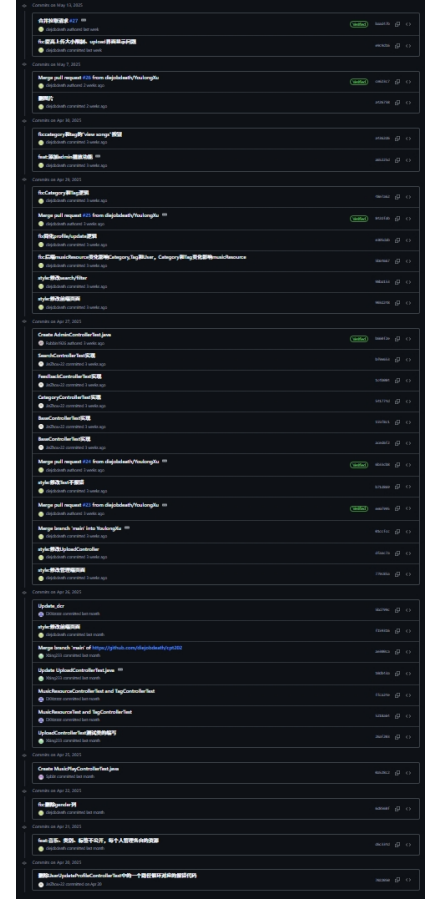
Appendix A: Evidences on Software Development Process



(a) Screenshots from Sprint 1 commits



(b) Screenshots from Sprint 2 commits



(c) Screenshots from Sprint 3 commits

Figure Apd.1: Sprint commits overview

Appendix B: Sequence Diagram UML about Sprint 1

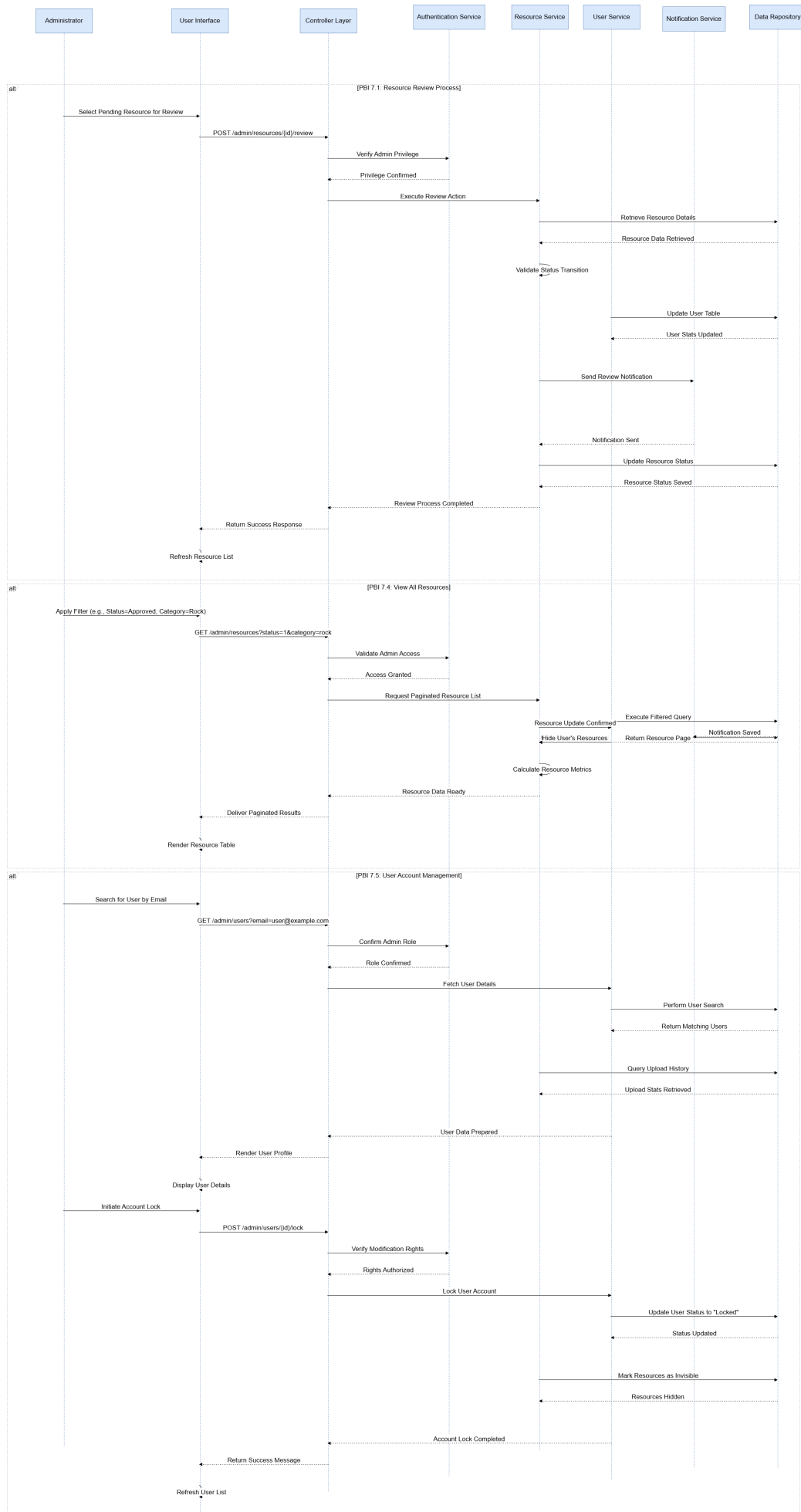


Figure Apd.2: Sequence Diagram UML about Sprint 1

Appendix C: ER Diagram

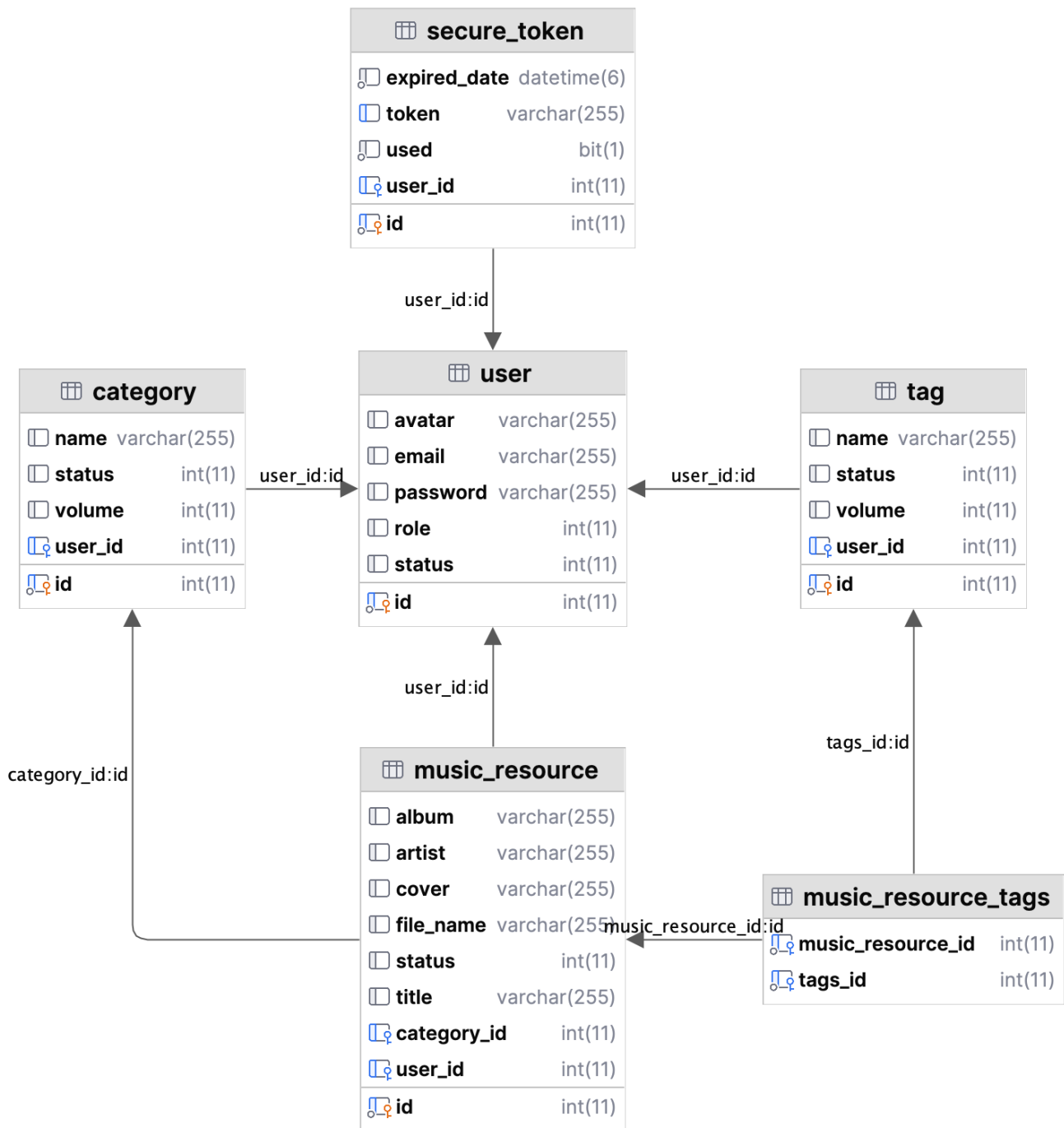


Figure Apd.3: ER Diagram

Appendix D: Main Code about Admin

```
1 <html lang="en" xmlns:th="http://www.thymeleaf.org">
2 <head>
3   <meta charset="UTF-8">
4   <meta name="viewport" content="width=device-width, initial-scale=1.0">
5   <title>Music Audit - Melodify Admin</title>
6   <link rel="stylesheet" th:href="@{/css/styles.css}">
7   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/
  css/all.min.css">
8 </head>
9 <body>
10 <div class="app" id="app">
11   <nav class="sidebar" :class="{ 'collapsed': !sidebarOpen }">
12     <button class="sidebar-toggle" @click="toggleSidebar">
13       <span></span>
14       <span></span>
15       <span></span>
16     </button>
17     <div class="logo">
18       <h1>Melodify</h1>
19     </div>
20     <div class="user-profile">
21       <a href="/admin/profile" class="profile-link">
22         
24         
26       </a>
27     </div>
28     <ul class="nav-links">
29       <li class="active"><a href="/admin/audit">Music Audit</a></li>
30       <li><a href="/admin/users">Users</a></li>
31       <li><a href="/admin/profile">Profile</a></li>
32     </ul>
33   </nav>
34   <main class="main-content">
35     <header class="top-bar">
36       <h1>Music Audit</h1>
37       <div class="top-bar-actions">
38         <form id="filterForm" action="/admin/audit" method="get">
39           <select name="status" class="custom-select" onchange="this.form.submit()"
40             >
41             <option value="">All Status</option>
42             <option value="0" th:selected="{selected}==0">Approved</option>
43             <option value="1" th:selected="{selected}==1">Pending</option>
44             <option value="2" th:selected="{selected}==2">Banned</option>
45           </select>
46         </form>
47         <button class="theme-toggle" @click="toggleTheme">
48           <i class="fas fa-moon"></i>
49         </button>
50       </div>
51     </header>
52     <section class="song-list">
53       <div class="song-grid">
54         <div th:each="music : {result}" class="song-card" th:data-id="{music.id}">
55           
57           
59           <h4 th:text="{music.title}">Title</h4>
60           <p th:if="{music.status}==0" style="color: lightgreen;font-weight: bold;
61             ">Normal</p>
62           <p th:if="{music.status}==1" style="color: yellow;font-weight: bold;">
```



```

59 Pending</p>
60 <p th:if="${music.status}==2" style="color: red;font-weight: bold;">
    Banned</p>
61 <p th:if="${music.status}==3" style="color: grey;font-weight: bold;">
    Deleted</p>
62 <p th:text="${music.artist}">Artist</p>
63 <p class="album" th:text="${music.album}">Album</p>
64 <audio th:src="@{/music/' + ${music.fileName}}" style="display: none;"
    ></audio>
65 <br>
66 <div class="upload-info">
    <span>Uploader: <span th:text="${music.user.name}">Uploader</span> </
    span>
67 </div>
68 <div class="status-badge" th:classappend="${'status-' + music.status}">
69 </div>
70 <div class="song-actions">
71     <button class="play-btn" onclick="playTrack(this)">
72         <i class="fas fa-play"></i>
73     </button>
74     <br>
75     <form th:if="${music.status == 1}" th:action="@{/admin/audit}" method
        ="post" style="display: inline;">
76         <input type="hidden" name="id" th:value="${music.id}">
77         <input type="hidden" name="status" th:value=0>
78         <button type="submit" class="btn btn-primary">
79             <i class="fas fa-check"></i> Approve
80         </button>
81     </form>
82     <form th:if="${music.status == 2}" th:action="@{/admin/audit}" method
        ="post" style="display: inline;">
83         <input type="hidden" name="id" th:value="${music.id}">
84         <input type="hidden" name="status" th:value=0>
85         <button type="submit" class="btn btn-primary">
86             <i class="fas fa-unlock"></i> Unban
87         </button>
88     </form>
89     <form th:if="${music.status == 0 || music.status == 1}" th:action="@
        {/admin/audit}" method="post" style="display: inline;">
90         <input type="hidden" name="id" th:value="${music.id}">
91         <input type="hidden" name="status" th:value=2>
92         <button type="submit" class="btn btn-danger">
93             <i class="fas fa-times"></i> Ban
94         </button>
95     </form>
96     <form th:if="${music.status == 2}" th:action="@{/admin/audit}" method
        ="post" style="display: inline;">
97         <input type="hidden" name="musicId" th:value="${music.id}">
98         <input type="hidden" name="status" th:value=3>
99         <button type="submit" class="btn btn-secondary">
100             <i class="fas fa-trash"></i> Delete
101         </button>
102     </form>
103     <form th:if="${music.status == 3}" th:action="@{/admin/audit}" method
        ="post" style="display: inline;">
104         <input type="hidden" name="musicId" th:value="${music.id}">
105         <input type="hidden" name="status" th:value=3>
106         <button type="button" class="btn btn-danger">
107             <i class="fas fa-ban"></i> Deleted
108         </button>
109     </form>
110 </div>
111 </div>
112 </div>
113 </section>
114 </main>
115

```

```

116 <div class="player">
117   <div class="now-playing">
118     
119     <div class="track-info">
120       <h4 id="currentTitle">No track selected</h4>
121       <p id="currentArtist">Select a song to play</p>
122     </div>
123   </div>
124   <div class="player-controls">
125     <button class="previous" onclick="previousTrack()">
126       <i class="fas fa-backward"></i>
127     </button>
128     <button class="play" onclick="togglePlay()">
129       <i class="fas fa-play"></i>
130     </button>
131     <button class="next" onclick="nextTrack()">
132       <i class="fas fa-forward"></i>
133     </button>
134   </div>
135   <div class="volume-control">
136     <i class="fas fa-volume-up"></i>
137     <input type="range" min="0" max="100" value="50" onchange="updateVolume(this.
      value)">
138   </div>
139 </div>
140 </div>
141
142 <script>
143   let currentAudio = null;
144   let currentPlayButton = null;
145   let currentVolume = 0.5;
146
147   function playTrack(button) {
148     const card = button.closest('.song-card');
149     const audio = card.querySelector('audio');
150     const icon = card.querySelector('i');
151
152     if (currentAudio && currentAudio !== audio) {
153       currentAudio.pause();
154       currentAudio.currentTime = 0;
155       if (currentPlayButton !== null) {
156         currentPlayButton.querySelector('i').className = 'fas fa-play';
157       }
158       document.querySelector('.player .play i').className = 'fas fa-play';
159     }
160
161     if (audio.paused) {
162       audio.volume = currentVolume;
163       audio.play();
164       icon.className = 'fas fa-pause';
165       document.querySelector('.player .play i').className = 'fas fa-pause';
166
167       // Update player info
168       document.getElementById('currentTitle').textContent = card.querySelector('h4').
        textContent;
169       document.getElementById('currentArtist').textContent = card.querySelector('p:nth
        -of-type(2)').textContent;
170       document.getElementById('currentCover').src = card.querySelector('img').src;
171
172       currentAudio = audio;
173       currentPlayButton = button;
174     } else {
175       audio.pause();
176       icon.className = 'fas fa-play';
177       document.querySelector('.player .play i').className = 'fas fa-play';
178     }
179   }

```

```

180
181 function togglePlay() {
182     if (currentPlayButton) {
183         playTrack(currentPlayButton);
184     }
185 }
186
187 function previousTrack() {
188     if (currentPlayButton) {
189         const currentCard = currentPlayButton.closest('.song-card');
190         const prevCard = currentCard.previousElementSibling;
191         if (prevCard) {
192             const prevButton = prevCard.querySelector('.play-btn');
193             playTrack(prevButton);
194         }
195     }
196 }
197
198 function nextTrack() {
199     if (currentPlayButton) {
200         const currentCard = currentPlayButton.closest('.song-card');
201         const nextCard = currentCard.nextElementSibling;
202         if (nextCard) {
203             const nextButton = nextCard.querySelector('.play-btn');
204             playTrack(nextButton);
205         }
206     }
207 }
208
209 function updateVolume(value) {
210     currentVolume = value / 100;
211     if (currentAudio) {
212         currentAudio.volume = currentVolume;
213     }
214 }
215
216 // Initialize theme
217 document.documentElement.setAttribute('data-theme', 'dark');
218 </script>
219 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
220 <script th:src="@{/js/scripts.js}"></script>
221 </body>
222 </html>

```

Listing 1: Admin.html

```

1
2 import com.cpt202.group7.model.Category;
3 import com.cpt202.group7.model.MusicResource;
4 import com.cpt202.group7.model.Tag;
5 import com.cpt202.group7.model.User;
6 import com.cpt202.group7.repository.MusicResourceRepo;
7 import com.cpt202.group7.repository.UserRepo;
8 import com.cpt202.group7.service.UserService;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11 import jakarta.servlet.http.HttpSession;
12 import org.springframework.beans.factory.annotation.Autowired;
13 import org.springframework.stereotype.Controller;
14 import org.springframework.ui.Model;
15 import org.springframework.util.CollectionUtils;
16 import org.springframework.util.FileCopyUtils;
17 import org.springframework.util.StringUtils;
18 import org.springframework.web.bind.annotation.*;
19
20 import java.io.File;
21 import java.io.FileInputStream;
22 import java.io.IOException;

```

```

23 import java.io.UnsupportedEncodingException;
24 import java.util.List;
25 import java.util.Set;
26
27 @Controller
28 @RequestMapping("/admin")
29 public class AdminController {
30
31     @Autowired
32     private UserRepo userRepo;
33
34     @Autowired
35     private MusicResourceRepo musicResourceRepo;
36
37     @Autowired
38     private UserService userService;
39
40     @GetMapping("")
41     public String getAdminPage(HttpSession session, Model m) {
42         User loginUser = (User) session.getAttribute("loginUser");
43         if (loginUser.getRole() == 0) {
44             return "redirect:/admin/audit";
45         }
46         return "redirect:/login";
47     }
48
49     @GetMapping("/users")
50     public String users(HttpSession session, @RequestParam(value = "status", required =
51         false) Integer status, Model m) {
52         User loginUser = (User) session.getAttribute("loginUser");
53         if (loginUser.getRole() == 0) {
54             m.addAttribute("avatar", loginUser.getAvatar());
55             if (status != null) {
56                 m.addAttribute("selected", status);
57                 m.addAttribute("users", userRepo.findByRoleAndStatus(1, status));
58             }
59             else {
60                 m.addAttribute("selected", null);
61                 m.addAttribute("users", userRepo.findByRole(1));
62             }
63             return "users";
64         }
65         return "redirect:/login";
66     }
67
68     // audit main page (with session verification)
69     @GetMapping("/audit")
70     public String getAuditPage(HttpSession session, @RequestParam(value = "status",
71         required = false) Integer status, Model m) {
72         User loginUser = (User) session.getAttribute("loginUser");
73         if (loginUser.getRole() == 0) {
74             m.addAttribute("avatar", loginUser.getAvatar());
75             if (status != null) {
76                 m.addAttribute("selected", status);
77                 m.addAttribute("result", musicResourceRepo.findByStatus(status));
78             }
79             else {
80                 m.addAttribute("selected", status);
81                 m.addAttribute("result", musicResourceRepo.findAll());
82             }
83             return "audit";
84         }
85         return "redirect:/login";
86     }
87
88     // handle lock user operations
89     @PostMapping("/audit")

```

```

88     public String postAuditPage(
89         Integer id,
90         Integer status,
91         HttpSession session,
92         Model m) {
93
94         // check if is admin login
95         User adminUser = (User) session.getAttribute("loginUser");
96         if (adminUser == null || adminUser.getRole() != 0) {
97             return "redirect:/login";
98         }
99
100        MusicResource musicResource = musicResourceRepo.findFirstById(id);
101        if (musicResource == null) {
102            m.addAttribute("error", "Music Resource Not Found");
103            return "redirect:/admin/audit";
104        }
105
106        if ((status == 0 || status == 1) && musicResource.getStatus() == 2) {
107            User uploadUser = musicResource.getUser();
108            uploadUser.setBanCount(uploadUser.getBanCount() - 1);
109        }
110
111        if (status == 2 && musicResource.getStatus() != 2) {
112            User uploadUser = musicResource.getUser();
113            uploadUser.setBanCount(uploadUser.getBanCount() + 1);
114            Category category = musicResource.getCategory();
115            if (category != null) {
116                category.setVolume(category.getVolume() - 1);
117            }
118            List<Tag> tags = musicResource.getTags();
119            if (!CollectionUtils.isEmpty(tags)) {
120                for (Tag tag : tags) {
121                    tag.setVolume(tag.getVolume() - 1);
122                }
123            }
124            musicResource.setCategory(null);
125            musicResource.setTags(null);
126        }
127
128        musicResource.setStatus(status);
129        musicResourceRepo.save(musicResource);
130        return "redirect:/admin/audit";
131    }
132
133    // handle lock user operations
134    @PostMapping("/users/lock")
135    public String lockUser(
136        Integer userId,
137        Model m,
138        HttpSession session) {
139
140        // check if is admin login
141        User adminUser = (User) session.getAttribute("loginUser");
142        if (adminUser == null || adminUser.getRole() != 0) {
143            return "redirect:/login";
144        }
145
146        // handle user status changes
147        User existingUser = userRepo.findFirstById(userId);
148        if (existingUser != null) {
149            // if is admin, cannot modify other admins
150            if (existingUser.getRole() == 0) {
151                m.addAttribute("error", "Cannot modify admins");
152                return "redirect:/admin/users";
153            }
154        }

```

```

155         existingUser.setStatus(2);
156         userRepo.save(existingUser);
157         return "redirect:/admin/users";
158     } else {
159         m.addAttribute("error", "User not found");
160         return "redirect:/admin/users";
161     }
162
163 }
164
165 @PostMapping("/users/unlock")
166 public String unlockUser(
167     Integer userId,
168     Model m,
169     HttpSession session) {
170
171     // check if is admin login
172     User adminUser = (User) session.getAttribute("loginUser");
173     if (adminUser == null || adminUser.getRole() != 0) {
174         return "redirect:/login";
175     }
176
177     // handle user status changes
178     User existingUser = userRepo.findFirstById(userId);
179     if (existingUser != null) {
180         // if is admin, cannot modify other admins
181         if (existingUser.getRole() == 0) {
182             m.addAttribute("error", "Cannot modify admins");
183             return "redirect:/admin/users";
184         }
185         existingUser.setStatus(0);
186         userRepo.save(existingUser);
187         return "redirect:/admin/users";
188     } else {
189         m.addAttribute("error", "User not found");
190         return "redirect:/admin/users";
191     }
192
193 }
194
195 @GetMapping("/profile")
196 public String viewProfile(Model model, HttpSession session) {
197     User user = (User) session.getAttribute("loginUser");
198     if (user.getRole() == 0) {
199         model.addAttribute("user", user); // Add user information to the model
200         model.addAttribute("avatar", user.getAvatar());
201         return "profileAdminPage"; // Return the view name
202     }
203     return "redirect:/login";
204 }
205
206 @RequestMapping(value="/view/uploads/**",method= {RequestMethod.GET})// a/b/c.gif
207 public void view(HttpServletRequest request, HttpServletResponse response) throws
208     UnsupportedEncodingException {
209     String fileName = request.getRequestURI();
210     fileName = fileName.substring(fileName.indexOf("uploads")+8);
211
212     String uploadDir = System.getProperty("user.dir") + "/src/main/resources/static/img"
213         ;
214
215     String ffname = new String(fileName.getBytes("GB2312"), "ISO-8859-1");
216
217     response.setContentType("application/force-download");
218     response.addHeader("Content-Disposition", "attachment;fileName=" + ffname);
219
220     File file = new File(uploadDir+File.separator+fileName);
221     try {

```

```

220         FileCopyUtils.copy(new FileInputStream(file), response.getOutputStream());
221     } catch (IOException e) {
222         return ;
223     }
224 }
225
226 // Update user profile information
227 @PostMapping("/profile/update")
228 public String updateProfile(@RequestBody User form, HttpSession session) {
229     User adminUser = (User) session.getAttribute("loginUser");
230     if (adminUser == null || adminUser.getRole() == 0) {
231         User user = userService.getUserById(form.getId());
232
233         // Update user profile information
234         user.setEmail(form.getEmail());
235         user.setAvatar(form.getAvatar());
236
237         // Save updated user information
238         userService.updateUserInfo(form.getId(), user);
239
240         // Redirect to the success page after profile update
241         return "redirect:/admin/profile";
242     }
243     return "redirect:/login";
244 }
245
246 }

```

Listing 2: AdminController.java

Appendix E: Main Code about Feedback

```
224 <html lang="en" xmlns:th="http://www.thymeleaf.org">
225 <head>
226   <meta charset="UTF-8">
227   <meta name="viewport" content="width=device-width, initial-scale=1.0">
228   <title>Profile - Melodify</title>
229   <link rel="stylesheet" th:href="@{/css/styles.css}">
230   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/
      css/all.min.css">
231 </head>
232 <body>
233 <div class="app" id="app">
234   <nav class="sidebar" :class="{ 'collapsed': !sidebarOpen }">
235     <button class="sidebar-toggle" @click="toggleSidebar">
236       <span></span>
237       <span></span>
238       <span></span>
239     </button>
240     <div class="logo">
241       <h1>Melodify</h1>
242     </div>
243     <div class="user-profile">
244       <a href="/admin/profile" class="profile-link">
245         
246         
247       </a>
248     </div>
249     <ul class="nav-links">
250       <li><a href="/admin/audit">Music Audit</a></li>
251       <li><a href="/admin/users">Users</a></li>
252       <li class="active"><a href="/admin/profile">Profile</a></li>
253     </ul>
254   </nav>
255
256   <main class="main-content">
257     <header class="top-bar">
258       <h1>Profile Settings</h1>
259       <div class="top-bar-actions">
260         <form action="/logout" method="POST" style="display: inline;">
261           <button type="submit" class="logout-btn">
262             <i class="fas fa-sign-out-alt"></i>
263             Logout
264           </button>
265         </form>
266         <button class="theme-toggle" @click="toggleTheme">
267           <i class="fas fa-moon"></i>
268         </button>
269       </div>
270     </header>
271
272     <div class="profile-container">
273       <div class="profile-header">
274         <div class="profile-avatar">
275           
276           
277           <label for="avatar-upload" class="avatar-upload-btn">
278             <span>Change Photo</span>
279             <input type="file" id="avatar-upload" name="avatar" accept="image/*" onchange="
                  previewAvatar(event)" style="display: none;">
280           </label>
281         </div>
282         <div class="profile-info">
```



```

283     <h2 th:text="${user.name}">Username</h2>
284     <p th:text="${user.email}">user@example.com</p>
285 </div>
286 </div>
287
288 <div class="profile-content">
289     <div class="profile-section">
290         <form class="profile-form" action="/profile/update" method="POST">
291             <input type="hidden" name="id" th:value="${user.id}">
292             <div class="form-group">
293                 <label for="username">Username</label>
294                 <input type="text" id="username" name="name" th:value="${user.name}">
295             </div>
296             <div class="form-group">
297                 <label for="email">Email</label>
298                 <input type="email" id="email" name="email" th:value="${user.email}">
299             </div>
300             <div class="form-group">
301                 <label for="current-password">Current Password</label>
302                 <input type="password" id="current-password" name="currentPassword">
303             </div>
304             <div class="form-group">
305                 <label for="new-password">New Password</label>
306                 <input type="password" id="new-password" name="newPassword">
307             </div>
308             <div class="form-group">
309                 <label for="password-confirm">Confirm Password</label>
310                 <input type="password" id="password-confirm" name="confirmPassword">
311             </div>
312             <div th:if="${updateMessage}" class="alert alert-success" style="color: green;
313                 margin-bottom: 20px;">
314                 <span th:text="${updateMessage}"></span>
315             </div>
316             <div th:if="${updateError}" class="alert alert-danger" style="color: red; margin
317                 -bottom: 20px;">
318                 <span th:text="${updateError}"></span>
319             </div>
320             <button type="submit" class="save-button">Save Changes</button>
321         </form>
322     </div>
323
324     <div class="profile-section">
325         <div class="feedback-section">
326             <h3>Send Feedback</h3>
327             <form class="feedback-form" action="/feedback" method="POST">
328                 <div class="form-group">
329                     <label for="feedback-type">Type</label>
330                     <select id="feedback-type" name="type" required>
331                         <option value="bug">Bug Report</option>
332                         <option value="feature">Feature Request</option>
333                         <option value="other">Other</option>
334                     </select>
335                 </div>
336                 <div class="form-group">
337                     <label for="feedback-message">Message</label>
338                     <textarea id="feedback-message" name="message" required rows="6"></textarea>
339                 </div>
340                 <div th:if="${feedbackMessage}" class="alert alert-success" style="color:
341                     green; margin-bottom: 20px; text-align: center;">
342                     <span th:text="${feedbackMessage}"></span>
343                 </div>
344                 <div th:if="${feedbackError}" class="alert alert-danger" style="color: red;
345                     margin-bottom: 20px; text-align: center;">
346                     <span th:text="${feedbackError}"></span>
347                 </div>
348                 <button type="submit" class="feedback-button">Submit Feedback</button>
349             </form>
350         </div>
351     </div>

```

```

346     </div>
347   </div>
348 </div>
349   </div>
350 </main>
351 </div>
352
353 <script>
354   function previewAvatar(event) {
355     const file = event.target.files[0];
356     if (file) {
357       const reader = new FileReader();
358       reader.onload = function(e) {
359         document.getElementById('avatar').src = e.target.result;
360         document.getElementById('defaultAvatar').src = e.target.result;
361       }
362       reader.readAsDataURL(file);
363     }
364   }
365 </script>
366 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
367 <script th:src="@{/js/scripts.js}"></script>
368 </body>
369 </html>

```

Listing 3: AdminProfile.html

```

248
249 import com.cpt202.group7.model.User;
250 import jakarta.servlet.http.HttpSession;
251 import org.springframework.stereotype.Controller;
252 import org.springframework.ui.Model;
253 import org.springframework.web.bind.annotation.GetMapping;
254 import org.springframework.web.bind.annotation.PostMapping;
255
256 @Controller
257 public class FeedbackController {
258     @GetMapping("/feedback")
259     public String getFeedbackPage(HttpSession session) {
260         User user = (User) session.getAttribute("loginUser");
261         if (user.getRole() == 0) {
262             return "redirect:/admin/profile";
263         }
264         return "redirect:/profile";
265     }
266
267     @PostMapping("/feedback")
268     public String feedback(String message, String type, HttpSession session, Model m) {
269         User user = (User) session.getAttribute("loginUser");
270         if (user.getRole() == 0) {
271             m.addAttribute("feedbackMessage", "Send feedback successfully");
272             m.addAttribute("user", user); // Add user information to the model
273             m.addAttribute("avatar", user.getAvatar());
274             return "profileAdminPage";
275         }
276         m.addAttribute("user", user);
277         m.addAttribute("avatar", user.getAvatar());
278         m.addAttribute("feedbackMessage", "Send feedback successfully");
279         return "profilePage";
280     }
281 }

```

Listing 4: FeedbackController.java

Appendix F : User Admin Sequence Diagram

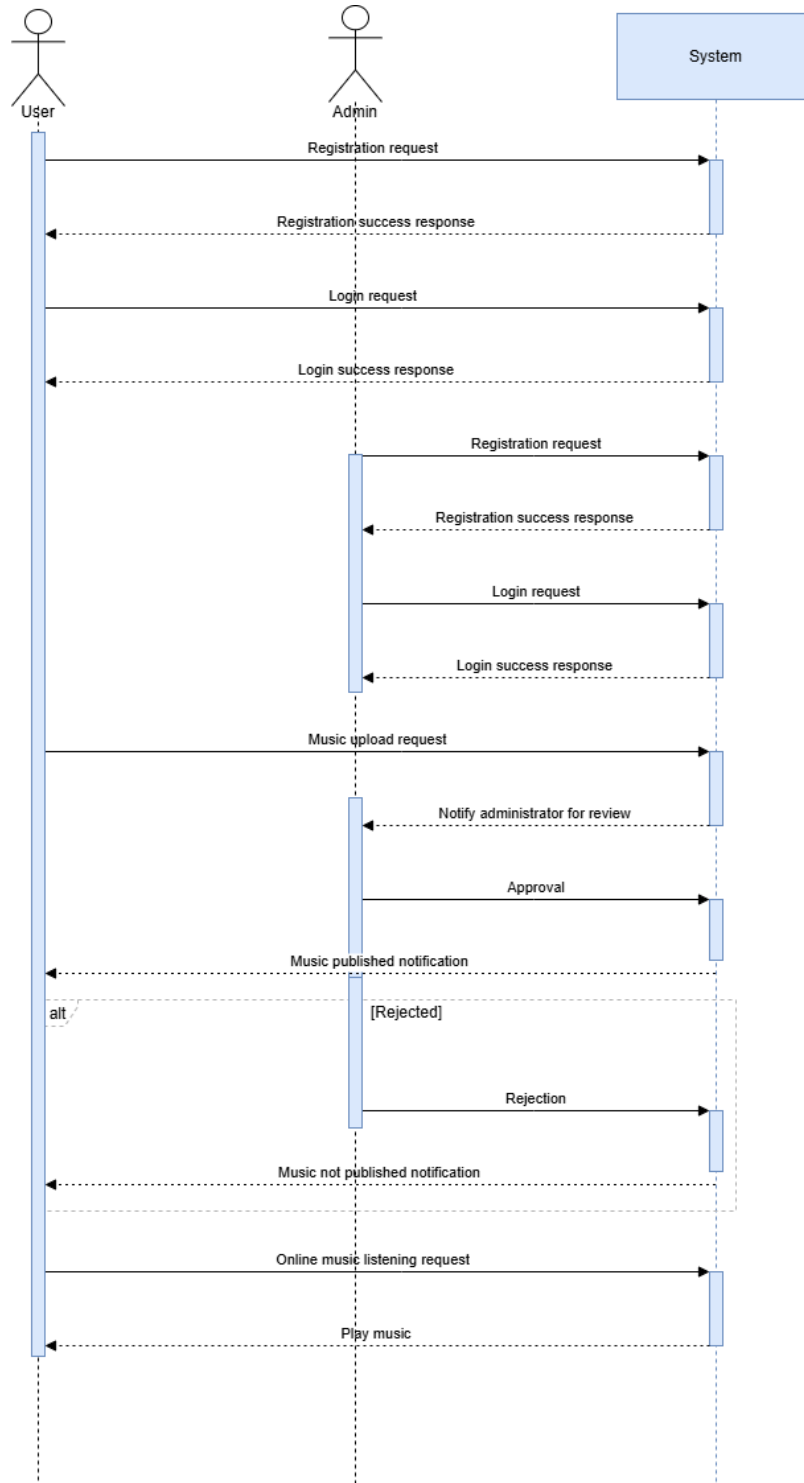


Figure Apd.4: Sequence Diagram For Music Resource Management

Appendix G: Secure encryption

user [cpt202@101.37.37.8]

🔍

📄

+

🔄

🔍

DDL

🔍

📄

📄

📄

CSV

📄

📄

🔍

🔍

WHERE

ORDER BY

	id	avatar	email	password	role	status
1	1	avatar_1_1745312702894.jpg	admin@123.com	\$2a\$12\$wx2hZhtv9f7yqDTwRhziKep4Pk.mK0lcAmB30DvbunVHK7ET:	0	
2	2	male3.png	Youlong.Xu22@student.xjtlu.edu.cn	\$2a\$12\$oTSB8shJX7.1W3UwQR7NK.LEx.gTqpjcuVlp9zzKlrnq2Duo	1	
3	3	male1.png	Zichen.Qiu22@student.xjtlu.edu.cn	\$2a\$12\$Mj0/AFWAgmplTC93MpjWMuy9rN3HAADHL7anQDko9NLqYtay:	1	
4	4	female1.png	123@qq.com	\$2a\$12\$Rdb6L1edSUDhLs5Ut0J7i0wtSG/xNXyV.S0CztSrvf0dJD2Si	1	
5	5	female2.png	1234@qq.com	\$2a\$12\$5spgHywMrk8w7BSCQR0eILvpNeGMtueKvxZ7dIqN3iL4ioL	1	
6	6	male2.png	Xuexin.Zhan22@student.xjtlu.edu.cn	\$2a\$12\$AuKZgH2sNPBn5w9AoANQnOUNy6JRHEUvjqZfEwQd/COYkM63:	1	
7	7	default-avatar.jpg	12345@qq.com	\$2a\$12\$V27aFrqdpWZhtKZJ617KS.B1AbQ6N1/01ZZHKWXInJtT2zRi:	1	
8	10	avatar_10_1745905034176.jpg	7bxyphg@steamXk.vip	\$2a\$12\$SdrXjXtH6D1eFkaDHA.4efsApjKdchcLe9ysFYeSudEPXK9.	1	
	14	default-avatar.jpg	Xinyi.Yang2202@student.xjtlu.edu.cn	\$2a\$12\$N0/frcb0qkLVmY62ESCIV0bbjxR7Fn.g48nNim06FHHMFbuHj	1	
10	30	default-avatar.jpg	xu.chen22@student.xjtlu.edu.cn	\$2a\$12\$QoA5mVg5SCuFGL2nu/DXuXc64kiPk.hGa6Z5jY6ADCqEkzbzi	1	

Figure Apd.5: Passwords secure encryption

Appendix H: CW1 PBIs and Artifacts

A.1 PBI 7.1: Admin Review of User-Uploaded Resources

User Story: As an admin, I want to review music resources uploaded by users so that I can ensure content compliance.

A.2 PBI 7.2: Block Unauthorized/Prohibited Content

User Story: As an admin, I want to block unauthorized or prohibited music resources so that I can maintain platform content quality.

A.3 PBI 7.3: Lock and Unlock User Accounts

User Story: As an admin, I want to lock and unlock user accounts so that I can manage violating users.

A.4 PBI 7.4: View All Music Resources

User Story: As an admin, I want to view all music resources so that I can comprehensively manage platform content.

A.5 PBI 7.5: Manage User Accounts

User Story: As an admin, I want to manage user accounts so that I can maintain the platforms user base.